

Run and test the Telegram Physical Goods Shop bot

Step by step. MIA can stay disabled while you test: leave MIA_RECIPIENT, MIA_PHONE and MIA_IBAN empty and the bot does not offer MIA at all.

1. Create a test bot

- 1 In Telegram open @BotFather, send /newbot and copy the token.
- 2 Get your numeric Telegram ID from @myidbot.
- 3 Use a separate test bot, not your real shop bot.

2. Configure

```
git clone https://github.com/aventurine-tech/Telegram-shop
cd Telegram-shop
git checkout development
cp .env.example .env
```

Edit .env and set:

```
TOKEN=<token from BotFather>
OWNER_ID=<your Telegram ID>
BOT_LOCALE=ro          # default language: ro, en or ru
POSTGRES_PASSWORD=<any strong password>
REDIS_PASSWORD=<any password>
ADMIN_PASSWORD=<anything except "admin">
SECRET_KEY=<python -c "import secrets; print(secrets.token_hex(32))">
MIA_RECIPIENT=
MIA_PHONE=
MIA_IBAN=
COD_ENABLED=1
DELIVERY_ENABLED=1
PICKUP_ENABLED=1
PICKUP_ADDRESS=Str. Test 1, Chisinau
CLEAN_CHAT=1           # 0 keeps the whole chat history
```

- ADMIN_PASSWORD and SECRET_KEY must be changed. In Docker the web panel is reachable, so the bot refuses to start while either keeps its default.
- Keep REDIS_PASSWORD non-empty when using the bundled Redis.
- Never copy .env.example over an existing .env: add new variables by hand.

3. Start it

```
docker compose up -d --build
docker compose logs -f bot
```

Wait for the log to show the bot started. The container applies the database migrations itself. If it exits, read the log: a startup error names the problem.

4. Update an existing installation

```
git pull
docker compose up -d --build
```

Then send /start to the bot once (it clears the recent chat history and brings back the Catalog / Cart / Profile keyboard).

5. Set up the shop (as owner)

- 1 Send /start from your own account. Pick a language; you get the owner role. The chat shows a short welcome line ("Welcome to UMBRA"), the main menu, and the Catalog / Cart / Profile keyboard at the bottom.
- 2 Admin panel, then Categories: create a top category (for example "Hookah tobacco"), then subcategories under it (answer the "Parent category" step; for example Classic and Intense). A category holds either subcategories or products.
- 3 Admin panel, then Goods: add a product to a subcategory: name, description, price, category, stock. The next step asks for an optional photo (send it as a file to keep the original quality) or Skip.
- 4 Weight options: on the product use "Add option" (label such as 50 g, price, stock). Each option has its own price and stock and shares the product picture.
- 5 Optional: open <http://localhost:9090/admin> and log in with ADMIN_USERNAME / ADMIN_PASSWORD. In the product form the Name is one field; the Options rows have a plus button: press it for Option 1, Option 2, Option 3, and fill option name, price and stock in each row.

Web accounts: log in as the first Admin, open My account and change the password, then Web accounts and create a Staff account. The Staff user does not see Web accounts and /admin/web-users is forbidden.

Roles: in the web panel open Roles, create a role and tick the permission tags (for example Basic access + Orders for a packer). The number is calculated for you.

Languages: use two accounts with different languages. Profile, then Language changes yours; the welcome line stays above the menu and the keyboard labels change.

6. Place a test order (as a customer)

Use a second Telegram account.

- 1 /start, then the category on the main menu (or Catalog at the bottom), then the subcategory and the product.
- 2 On a product with options, pick the weight on the card; price and stock follow the option.
- 3 Add to cart, open the cart (Cart at the bottom), change the quantity, then check out.
- 4 Choose Delivery or Pickup, then enter name, phone, address (delivery only) and an optional comment.
- 5 The payment step should offer only cash on delivery/pickup. Confirm the order.

7. Check what should happen

- Stock: the option's stock drops by the quantity ordered.
- Staff alert: your owner account receives a "new order" message with an open-order button.
- My orders: the customer sees the order under Profile, then My orders.
- Order walk-through in the bot: Admin panel, Orders. Confirm, Mark shipped, Complete. The customer gets a message at each step.
- Order walk-through in the web panel: Orders, open an order: only the buttons that fit its state are shown (a new cash order: Confirm order and Cancel; confirmed: Mark as shipped, Mark as completed, Cancel; closed orders show none).
- Cash: completing the order marks the cash as collected.
- Cancel: place a second order and cancel it from the admin side. The stock comes back.
- Self-cancel: place a third order and cancel it as the customer from My orders.
- Out of stock: order the last units; the card shows out of stock with a notify-me button. Restocking messages that user.

- Clean chat: while you navigate, the chat keeps only the current screen; your own messages disappear after they are handled.

8. Import the UMBRA catalog (optional, once)

The catalog crawled from umbramd.com (54 products, 73 weight options, 3 languages, pictures) is in `scripts/umbramd/`.

- 1 Copy `scripts/umbramd/prices.template.csv` to `data/prices.csv` and fill in the price per weight (MDL).
- 2 Dry run: `docker compose exec bot python -m scripts.import_catalog scripts/umbramd/catalog.json --prices data/prices.csv --dry-run`
- 3 Import: the same command without `--dry-run`. Add `--top-category "Premium hookah tobacco"` to reuse an existing empty top category.
- 4 Stock is 0 for everything: set it per option. The import never changes existing items, so it can be repeated.

9. Optional extras

- Referral: set `REFERRAL_PERCENT=10`, restart the bot, and use another account's referral link. Completing that account's order credits the referrer's balance.
- Balance: Admin, Users, open a customer and add balance. The customer can then use it at checkout.
- Language: change `BOT_LOCALE`, then `docker compose restart bot`.

Handy commands

<code>docker compose logs -f bot</code>	<code># live log</code>
<code>docker compose restart bot</code>	<code># after editing .env</code>
<code>docker compose down</code>	<code># stop</code>
<code>docker compose down -v</code>	<code># stop and wipe the database</code>

Button labels may differ slightly from the names used here, because they come from the translation files. If a step fails, check the bot log first.